2026 NowCoder Summer Multi-School Training #8

Problem List

A	14 Minesweeper Variants
B	Deep Finesse
C	Fraction on a Ring
D	Sea, you & copriMe II
E	Prefix Codes
F	Tree shifts there
G	Multiplication
H	It's Magic, Not a Trick!
I	Bridge VI
J	Oni's Ring
K	Al Fine II
L	Shattered Minimum
M	KV Cache

This problem set should contain 13 problems on 30 numbered pages.

Please inform a runner immediately if your problem set is incomplete.

12 August, 2026

Hosted by

牛客竞赛
AC.NOWCODER.COM

Presented by

Problem A. 14 Minesweeper Variants

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

Silver Wolf has found a new game to play between Stellaron Hunter missions: *14 Minesweeper Variants*.

Naturally, she hacked the game before finishing the tutorial. She now has the internal level files, including the exact location of every mine. This should have made the game trivial.

Unfortunately, she selected **Expert Mode**.

In Expert Mode, knowing that a cell is safe is not enough. Before every click, the cell must be provably safe using only the information currently visible to an honest player. A move based only on the hacked minefield is illegal. After making no progress for an embarrassingly long time, Silver Wolf decides to do the reasonable thing: write a program that produces a complete legal sequence of clicks.

You are given a hacked internal description of the board. It contains the true minefield and the future behavior of every safe cell:

- `#` is a mine.
- `O` is an ordinary safe cell that is already open. It displays the number of mines in its eight neighboring cells.
- `Q` is a safe cell that is already open, but displays `?` instead of a number.
- `.` is a closed safe cell. When clicked, it displays its ordinary adjacent-mine number, namely, the total number of mines among its eight surrounding cells.
- `?` is a closed safe cell. When clicked, it displays `?` and gives no numeric clue.

Two cells are neighboring if they share a side or a corner.

The number of mines, equal to the number of `#` characters in the file, is known from the beginning.

The file contains more information than an honest player can see. In particular, all unopened `#`, `.`, and `?` cells look identical before they are clicked. Their symbols in the file may be used to simulate what will actually happen after a click, but may **not** be used to justify that click.

At any moment, consider every candidate mine placement satisfying all of the following conditions:

1. It contains exactly the known total number of mines.
2. Every currently open cell is safe.
3. Every numeric clue currently visible has exactly that many neighboring mines.

An open `Q` or `?` cell only tells the player that this cell is safe; it imposes no condition on its neighbors.

An unopened cell is **certainly safe** if it is not a mine in any candidate mine placement. You may click a cell only when it is currently certainly safe.

After a legal click, the actual level file determines what is revealed:

- clicking a `.` adds its adjacent-mine number to the visible information;
- clicking a `?` only reveals that the cell is safe and displays `?`.

Your goal is to open every safe cell. Cells marked `O` or `Q` are already open and must not appear in the output.

If several complete legal orders exist, output any of them. If it is impossible to open every safe cell using legal clicks, output -1.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \le n, m \le 6$) "— the number of rows and columns.

Each of the following n lines of each test case contains a string of length m over the characters `#`, `O`, `Q`, `.` and `?`.

Output

For each test case, output one answer.

If no complete legal sequence exists, output a single integer -1.

Otherwise, first output an integer k, followed by k lines. The i-th of these lines contains two integers r_i and c_i, the 1-indexed row and column of the i-th cell to click. It is possible that $k = 0$.

Example

standard input	standard output
4 1 3 O.# 1 3 Q.# 1 3 O?# 5 5 ###O# Q.#?O ?.... #..## .Q##?	1 1 2 -1 1 1 2 11 3 4 3 5 3 3 4 3 2 2 2 4 3 2 4 2 5 1 3 1 5 5

Note

In the first test case, the open ordinary cell displays 0, so cell $(1, 2)$ is certainly safe.

In the second test case, `Q` provides no numeric clue. There is one mine, and either unopened cell could contain it, so no legal click is available.

In the third test case, the open ordinary cell again proves that cell $(1, 2)$ is safe. Clicking it reveals `?`, but this is enough to finish because every safe cell is now open.

Almost every test case in this problem comes from a real level of *14 Minesweeper Variants*. Therefore, if the contest is becoming unbearable, you may first spend a little money on the game, play through the relevant levels, and hard-code all the answers.

Problem B. Deep Finesse

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

Ushio and Fuuko are locked in a battle of wits over an immense universe of numbers: all the integers from 1 up to 10^{100}. The duel begins with Ushio carefully selecting exactly n distinct numbers from this pool; Fuuko receives every remaining number. Then, over the course of n rounds, the two reveal their choices one by one. In each round, Fuuko acts first, displaying a number from her own vast collection, and Ushio must immediately answer with one of her remaining numbers. Here lies the perilous rule: if Fuuko's number is strictly smaller than Ushio's response, Fuuko seizes victory on the spot. Only when Ushio's number is less than or equal to Fuuko's does the round pass safely — both numbers are discarded and the game proceeds. To emerge triumphant, Ushio must endure all n rounds without ever allowing Fuuko to play a smaller number.

To make matters worse, Fuuko imposes an extra handicap before the game even starts. She hands Ushio a set of m specific numbers and declares that Ushio's starting hand must include every single one of them. Ushio now ponders her chances: among all possible hands of size n that contain these m forced numbers, how many can guarantee her a winning strategy, no matter how cleverly Fuuko chooses her moves? The answer may be monstrously large, so she asks you to compute it modulo 998244353.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 1000$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \le m \le n \le 5000$).

The second line of each test case contains m integers $a_1, a_2, \ldots, a_m$ ($1 \le a_i \le 10^9$) "— the specific numbers that Ushio must include in her hand. It is guaranteed that a contains no duplicates.

It is guaranteed that the sum of n over all test cases does not exceed 5000.

Output

For each test case, output one integer "— the number of Ushio's starting hands that can guarantee her a winning strategy modulo 998244353.

Example

standard input	standard output
3 1 1 2 2 1 2 6 3 1 4 5	0 1 34

Note

In the first test case, Ushio's hand is fixed to $\{2\}$. If Fuuko plays 1, Ushio must respond with 2, and since $1 < 2$, Fuuko wins immediately. Therefore, the answer is 0.

In the second test case, Ushio's hand must be $\{1, 2\}$ to guarantee a win. Therefore, the answer is 1.

This page is intentionally left blank.

Problem C. Fraction on a Ring

Input file:	standard input
Output file:	standard output
Time limit:	3 seconds
Memory limit:	1024 megabytes

Just as Soy typed the final sentence in the chat, an idea struck him: every residue class modulo a prime P has a unique integer representative in $[0, P)$. Dividing that representative by P yields a number in $[0, 1)$, so it can be interpreted as a probability. Thus, the fraction $\frac{a}{b}$ acquires a curious modular counterpart, $\frac{ab^{-1} \bmod P}{P}$. Soy immediately wondered: for how many choices is the original fraction larger than this "probability"?

You're given an integer n and a prime number P.

Find the number of distinct integer pairs (a, b) such that:

1. $1 \le a < b \le n$;
2. $\frac{a}{b} > \frac{ab^{-1} \bmod P}{P}$.

Because the answer can be very large, output it modulo 998244353.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 100$). The description of the test cases follows.

The only line of each test case contains two integers n and P ($2 \le n < P \le 10^{18}$, P is a prime number).

Output

For each test case, output one integer "— the number of integer pairs that satisfy the constraints modulo 998244353.

Example

standard input	standard output
3 4 5 64 97 998244353 1000000007	1 932 782525253

Note

In the first test case, all possible pairs are enumerated in the following table.

a/b		X/P
$1/2$	$<$	$3/5$
$1/3$	$<$	$2/5$
$1/4$	$<$	$4/5$
$2/3$	$<$	$4/5$
$2/4$	$<$	$3/5$
$3/4$	$>$	$2/5$

Problem D. Sea, you & copriMe II

Input file:	standard input
Output file:	standard output
Time limit:	3 seconds
Memory limit:	1024 megabytes

You're given an integer array a of length n. Define $f(l, r)$ as:

$$f(l,r) = \begin{cases} 1 & \exists\, p,q,s,t \in \{l, l+1, \ldots, r\}: \\ & |\{p,q,s,t\}| = 4, \quad \gcd(a_p, a_q) = \gcd(a_s, a_t) = 1 \\ 0 & \text{otherwise.} \end{cases}$$

In other words, $f(l, r) = 1$ if and only if there exist four distinct indices p, q, s, t such that $l \le p, q, s, t \le r$ and $\gcd(a_p, a_q) = \gcd(a_s, a_t) = 1$.

You're given q queries. Each query contains an interval $[l, r]$, and you need to compute:

$$\sum_{i=l}^{r} \sum_{j=i}^{r} f(i, j).$$

Input

The first line of each test contains two integers n and q ($1 \le n, q \le 5 \cdot 10^5$).

The second line of each test contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^6$).

Each of the following q lines of each test contains two integers l and r ($1 \le l \le r \le n$).

Output

For each query, output one integer "— the answer to the query.

Example

standard input	standard output
8 6 6 10 15 7 14 21 35 11 1 4 1 5 2 6 2 8 3 8 1 8	0 2 2 5 1 9

This page is intentionally left blank.

Problem E. Prefix Codes

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

In this problem, a **codeword** is a non-empty string, and a **codebook** is a non-empty set of distinct codewords.

A codebook is called a **prefix code** if, for every two distinct codewords x and y in the codebook, neither x is a prefix of y nor y is a prefix of x.

Soy is given two non-empty strings s and p. He wants to construct a prefix code using substrings of s as codewords. However, he does not want p to occur in any codeword.

More formally, a string x is an **eligible codeword** if all of the following conditions hold:

- x is non-empty;
- x is a substring of s;
- p is not a substring of x.

Substrings are distinguished only by their contents. If the same string occurs at several positions in s, it is still considered only one eligible codeword.

A **valid codebook** is a prefix code consisting entirely of eligible codewords. In other words, it is a non-empty set C of eligible codewords such that, for every two distinct strings $x, y \in C$, neither string is a prefix of the other.

Two valid codebooks are considered different if one of them contains a codeword that the other does not.

Find the number of different valid codebooks Soy can construct. Since the answer may be very large, output it modulo 998244353.

Input

The first line of each test contains one string s ($1 \le |s| \le 5 \cdot 10^5$).

The second line of each test contains one string p ($1 \le |p| \le 5 \cdot 10^5$).

It is guaranteed that both strings consist only of lowercase Latin letters.

Output

Output one integer ”— the number of different valid codebooks modulo 998244353.

Example

standard input	standard output
aba ba	5

Note

In the sample test, the valid codebooks are: {a}, {b}, {ab}, {a, b}, and {b, ab}.

This page is intentionally left blank.

Problem F. Tree shifts there

Input file:	standard input
Output file:	standard output
Time limit:	4 seconds
Memory limit:	1024 megabytes

On the Moon, Kagari spends her time studying an enormous collection of possibilities. She explains her discoveries in an extremely compressed language: a few of her words may contain enough information to overwhelm an ordinary human mind.

Kotarou, unwilling to admit defeat, keeps using his ability to accelerate his thoughts and follow her explanations. This has ended badly often enough that Kagari decides to measure his reaction speed before telling him anything more. Naturally, simplifying her language is not among the options she considers.

For this challenge, Kagari uses a World Tree with n vertices, numbered from 1 to n. The tree is undirected and unweighted.

Each query is described by two vertices u and v. Kagari temporarily removes every edge on the unique simple path from u to v. The World Tree becomes a forest consisting of several connected components.

The **diameter** of a connected component is the maximum distance between any two of its vertices, where distance is measured by the number of edges on the path between them. In particular, the diameter of a component containing one vertex is 0.

For every query, Kotarou must find the sum of the diameters of all resulting connected components.

The queries are **independent**. After each query, all removed edges are restored before the next query begins. If $u = v$, the path contains no edges, so the World Tree is not changed.

Input

The first line of each test contains two integers n and q ($1 \le n, q \le 2 \cdot 10^5$).

The following $n - 1$ lines of each test describe $n - 1$ edges. The i-th line contains two integers a_i and b_i ($1 \le a_i, b_i \le n$), denoting an undirected edge between vertices a_i and b_i. It is guaranteed that these edges form a tree.

The following q lines of each test describe q queries. The i-th line contains two integers u_i and v_i ($1 \le u_i, v_i \le n$).

Output

For each query, output one integer ”— the sum of the diameters of all connected components obtained after removing the corresponding edges.

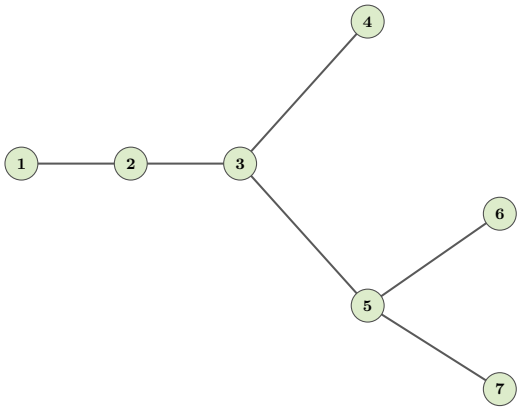
Example

standard input	standard output
7 5 1 2 2 3 3 4 3 5 5 6 5 7 1 4 6 7 3 3 2 5 1 7	2 3 4 4 2

Note

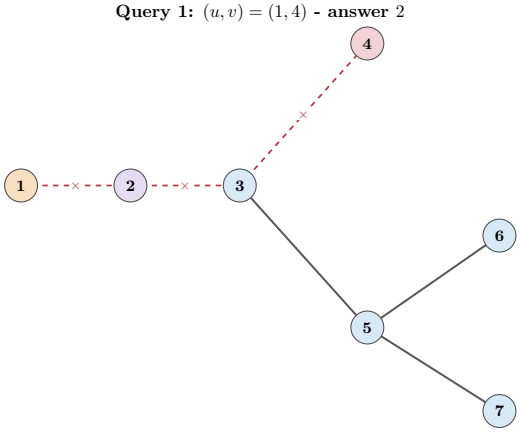
The explanation of the sample can be seen below.

Original tree - diameter 4



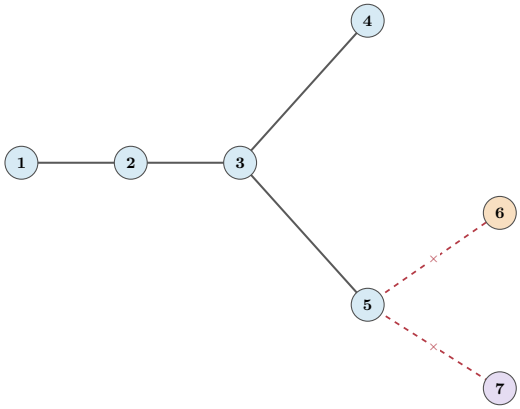
Edges: $1-2$, $2-3$, $3-4$, $3-5$, $5-6$, $5-7$.

Query 1: $(u, v) = (1, 4)$ **- answer** 2



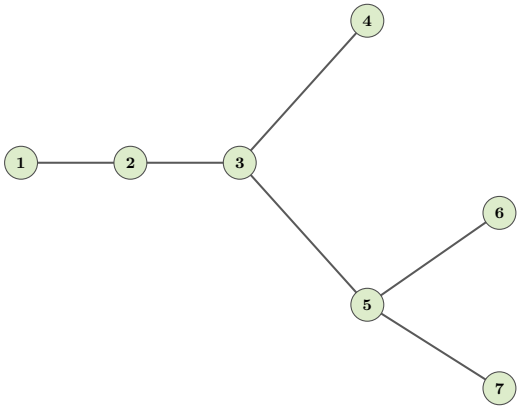
Component diameters: $0, 0, 0, 2$. Sum: 2.

Query 2: $(u, v) = (6, 7)$ **- answer** 3



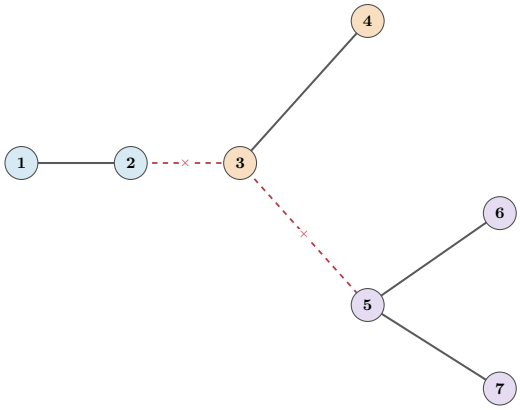
Component diameters: $3, 0, 0$. Sum: 3.

Query 3: $(u, v) = (3, 3)$ **- answer** 4



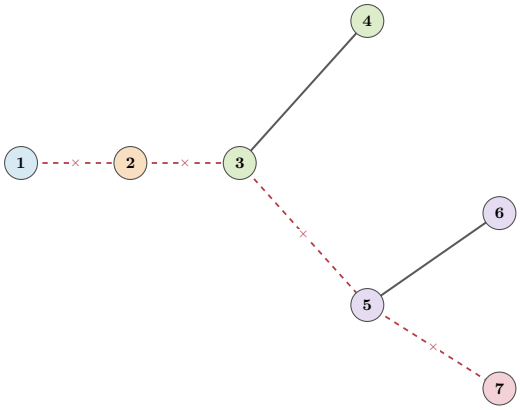
No edge is removed. Component diameter: 4.

Query 4: $(u, v) = (2, 5)$ **- answer** 4



Component diameters: $1, 1, 2$. Sum: 4.

Query 5: $(u, v) = (1, 7)$ **- answer** 2



Component diameters: $0, 0, 1, 1, 0$. Sum: 2.

This page is intentionally left blank.

Problem G. Multiplication

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

In the magical realm of Mozzarella, a summoning ritual went spectacularly wrong. Brownie intended to call forth the legendary sorceress Galette in her full glory, but instead only her head materialized. Unfazed, the floating head declared, "A head is all one needs! I shall prove that my mind alone surpasses your entire bodies."

To demonstrate her intellectual might, Galette made a bold claim:

"Consider any two positive integers. If you tell me just the first a digits of one number and the first b digits of the other, I can infallibly determine the first c digits of their product! The unseen lower digits are as irrelevant as a body to a genius."

Orangette, the academy's sharpest mathematician, immediately sensed a flaw. But merely pointing out the fallacy wasn't enough. Galette demanded a concrete counterexample: "If you're so clever, show me two different pairs of numbers whose leading parts agree, yet whose products disagree in the leading part!"

Orangette struggled to construct such an example on the fly, and now she turns to you for help.

Formally, you're given three positive integers a, b, and c. Construct two pairs of positive integers (x_1, y_1) and (x_2, y_2) with the following properties:

- In ordinary decimal notation (without leading zeros), x_1 and x_2 each have at least a digits, and their first a digits are identical.
- Similarly, y_1 and y_2 each have at least b digits, and their first b digits are identical.
- The products $x_1 \cdot y_1$ and $x_2 \cdot y_2$ each have at least c digits, but their first c digits are different.

Input

The only line of each test contains three integers a, b, and c ($1 \le a, b \le 10^5$, $1 \le c < a + b - 1$).

Output

Output four integers x_1, y_1, x_2, and y_2 ($10^{a-1} \le x_1, x_2 < 10^{5 \cdot 10^5}$, $10^{b-1} \le y_1, y_2 < 10^{5 \cdot 10^5}$) that satisfy the requirements. None of these integers can have leading zeros. It can be shown that a solution always exists under the given constraints.

Example

standard input	standard output
2 2 2	1704 1313 1789 1346

Note

The first 2 digits of $1704 \times 1313 = 2237352$ are 22. The first 2 digits of $1789 \times 1346 = 2407994$ are 24. Therefore, this is a reasonable construct.

This page is intentionally left blank.

Problem H. It's Magic, Not a Trick!

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

Nyeh... what a pain, but a real mage never gives up.

Himiko Yumeno, the self-proclaimed Ultimate Mage, has placed n enchanted talismans in a row before her. The i-th talisman initially holds a_i units of magical energy, and each a_i is a positive integer. Lately, she has been studying a dusty old grimoire and has deciphered a peculiar spell that manipulates energy in a very specific way.

To cast the spell once, she must perform two steps in exact order:

1. Infusion: She picks any talisman and channels exactly 1 unit of her own energy into it, increasing that talisman's energy by 1.
2. Discharge: She picks any talisman that, right after the infusion, possesses at least x units of energy. She then unleashes a brilliant burst that consumes exactly x units from that talisman, reducing its energy by x. She may choose the same talisman for both steps, or a different one; the spell is flexible in that regard.

There's a critical restriction, however: if after the infusion there is no talisman with at least x energy, the discharge cannot be performed. In that case, the whole spell simply fizzles out and nothing happens, and the energy of the talisman she tried to infuse remains unchanged.

Himiko may attempt this spell as many times as she likes, provided each attempt is successful and all talismans keep nonnegative energy. Curious about the limits of her craft, she wonders: by choosing where to infuse and discharge each time, what is the minimum possible sum of energy remaining across all talismans? She'd much rather take a nap than do the arithmetic herself, so she turns to you, her trusty assistant, for help. Because the answer can be very large, you only need to output it modulo 998244353.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and x ($1 \le n \le 2 \cdot 10^5$, $1 \le x \le 10^9$).

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 10^{18}$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output one integer "— the minimum possible sum of remaining energy across all talismans modulo 998244353.

Example

standard input	standard output
3 3 3 1 1 1 1 4 3 4 10 30 8 7 6	3 0 6

Note

In the first test case, Himiko cannot perform any spell.

In the second test case, Himiko can both infuse and discharge on talisman 1 to let its energy become 0.

Problem I. Bridge VI

Input file:	standard input
Output file:	standard output
Time limit:	1 second
Memory limit:	1024 megabytes

Soy takes part in a competition with $2n$ participants, which has now reached the final round. The participants are numbered from 1 to $2n$. In the final round, for each $i = 1, 2, \ldots, n$, participant $2i - 1$ competes against participant $2i$. In each match, m points are distributed between the two participants in any way, provided that each receives an amount of a non-negative **real number**.

Before the final round, participant i has an initial score a_i. The final score of a participant is their initial score plus the points they earn in this round.

Soy is participant 1. He wants to know his best possible and worst possible final standing. Specifically, considering all valid ways to distribute the m points within each pair, he wants to know:

- the minimum possible number of participants whose final score is **strictly greater** than his,
- the maximum possible number of participants whose final score is **strictly greater** than his.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \le n \le 2 \cdot 10^5$, $1 \le m \le 10^9$).

The second line of each test case contains $2n$ integers $a_1, a_2, \ldots, a_{2n}$ ($1 \le a_i \le 10^9$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output two integers ”— the minimum and maximum possible number of participants whose final score is strictly greater than Soy's.

Example

standard input	standard output
3 2 3 4 5 6 7 2 1 6 6 6 6 6 6 9 9 8 2 4 4 3 5 3 3 1 4	1 3 0 3 0 5

This page is intentionally left blank.

Problem J. Oni's Ring

Input file:	standard input
Output file:	standard output
Time limit:	2 seconds
Memory limit:	1024 megabytes

Kamiyama Shiki is investigating an old legend about an oni who sealed its memories inside a ring of black and white stones.

Shiki represents each black stone by 0 and each white stone by 1. Thus, the ring forms a circular binary string s.

To examine the ring, Shiki chooses a starting position i and reads k consecutive characters clockwise, wrapping around to the beginning whenever she reaches the end of the string. For a binary string x of length k, if the string read from position i is equal to x, we say that x appears at position i of s.

The number of occurrences of x in s is the number of different starting positions at which x appears. For example, `10` appears exactly twice in the circular binary string `00110011`.

Unfortunately, before Shiki could finish her investigation, the oni's ring disappeared. All that remains is her notebook, which contains 2^k non-negative integers $c_0, c_1, \ldots, c_{2^k-1}$.

For every integer x with $0 \le x < 2^k$, write x as a binary string of length exactly k, adding leading zeroes when necessary. Shiki's notes state that this binary string appeared exactly c_x times in the missing ring.

A circular ring has no distinguished starting point. Therefore, two circular binary strings are considered the same if one can be obtained from the other by rotation. For example, `1010` and `0101` represent the same circular binary string.

Help Shiki reconstruct the oni's lost seal by finding the number of distinct circular binary strings consistent with all the occurrence counts in her notebook.

Since the answer may be large, output it modulo 998244353.

Input

The first line of each test contains one integer k ($1 \le k \le 9$).

The second line of each test cases contains 2^k non-negative integers $c_0, c_1, \ldots, c_{2^k-1}$.

It is guaranteed that the sum of c_i is at least 1 and does not exceed $2 \cdot 10^5$.

Output

Output one integer "— the number of distinct valid circular binary strings modulo 998244353.

Examples

standard input	standard output
1 2 2	2
2 1 1 1 1	1
2 0 1 0 0	0

Note

In the first sample test, the valid circular binary strings are `0011` and `0101`.

In the second sample test, the only valid circular binary string is `0011`.

This page is intentionally left blank.

Problem K. AI Fine II

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

For Christmas, the twin sisters Tortinita and Arietta each receive a Christmas tree. They want to choose some of n candidate ornaments to decorate their trees.

Tortinita's tree has a vertices, and Arietta's tree has b vertices. Both trees are rooted at vertex 1.

For every ornament, the sisters have independently decided where it would be hung on their trees. Ornament i would be hung at vertex x_i of Tortinita's tree and at vertex y_i of Arietta's tree. The sisters must choose the same set of ornaments: an ornament is either placed on both trees or on neither tree.

The branches of the Christmas trees can support only limited weight. Every vertex of either tree has a load limit: the number of selected ornaments hung at that vertex or anywhere in its subtree must not exceed this limit.

Every ornament also has a beauty value. The sisters want to choose exactly k distinct ornaments while respecting every load limit on both trees. Find the maximum possible sum of beauty values.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 20$). The description of the test cases follows.

The first line of each test case contains three positive integers n, a, and b, and one non-negative integer k ($0 \le k \le n$).

The next a lines of each test case describe Tortinita's tree. The i-th line contains two integers p_i, c_i ($0 \le c_i \le n$) "— the parent and the load limit of vertex i, respectively. It is guaranteed that $p_1 = 0$. For every $i > 1$, it is guaranteed that $1 \le p_i < i$.

The next b lines of each test case describe Arietta's tree in the same format.

The next n lines of each test case describe the ornaments. The i-th line contains three integers x_i, y_i, w_i ($1 \le x_i \le a$, $1 \le y_i \le b$, $0 \le w_i \le 10^9$) "— its position on Tortinita's tree, its position on Arietta's tree, and its beauty value, respectively.

It is guaranteed that the sum of $n + a + b$ over all test cases does not exceed 3000.

Output

For each test case, output the maximum possible sum of beauty values. If it is impossible to choose exactly k ornaments, output -1.

Example

standard input	standard output
2 5 3 3 3 0 3 1 1 1 2 0 3 1 2 1 1 2 2 8 2 3 7 3 2 6 3 2 5 3 3 4 2 1 2 2 0 2 0 1 1 2 1 2 10 1 2 9	18 -1

Note

In the first test case, choosing ornaments $1, 3, 5$ gives a total beauty of $8 + 6 + 4 = 18$. Two selected ornaments are located in the subtree of vertex 3 of Tortinita's tree, and two are located in the subtree of vertex 2 of Arietta's tree. Both corresponding load limits are met exactly, and all other limits are also respected.

In the second test case, the root of Arietta's tree has a load limit of 1, so it is impossible to choose two ornaments.

Problem L. Shattered Minimum

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

During their relaxation in an abandoned facility, Chito and Yuuri find a terminal that is still working. While Chito still checks the terminal carefully, Yuuri has already pressed the button. n blocks are then lined up on the screen, and their numbers form a permutation p of length n. Yuuri is highly interested in the terminal, so the duo decide to play a game based on the terminal's manual.

The terminal can copy an arbitrary interval $[l, r]$ of p, forming an independent segment. There are m segments initially. The game is played in turns, with Chito going first. In each turn, the current player must:

1. Choose a segment, and the terminal will remove the block with the minimum number. The segment is then divided into two segments.
2. Choose one of the following:
   - Keep the left segment, and discard the right segment.
   - Keep the right segment, and discard the left segment.
   - Keep both segments.

After each turn, all empty segments are discarded. Both players are allowed to keep empty segments in Step 2, so that the entire segment is discarded.

The player who cannot operate first loses. Suppose both Chito and Yuuri play optimal strategies; determine who wins the game.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 2 \cdot 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and m ($1 \le n, m \le 2 \cdot 10^5$).

The second line of each test case contains n integers $p_1, p_2, \ldots, p_n$ ($1 \le p_i \le n$, $p_i \ne p_j$).

Each of the following m lines of each test case contains two integers l and r ($1 \le l \le r \le n$).

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

It is guaranteed that the sum of m over all test cases does not exceed $2 \cdot 10^5$.

Output

For each test case, output one string "— `Chito` if Chito wins, and `Yuuri` otherwise.

Example

standard input	standard output
4 1 1 1 1 1 1 2 1 1 1 1 1 4 1 2 1 3 4 1 4 3 2 2 1 3 1 3 1 1	Chito Yuuri Yuuri Chito

Problem M. KV Cache

Input file:	`standard input`
Output file:	`standard output`
Time limit:	2 seconds
Memory limit:	1024 megabytes

Soy is running a Large Language Model service. To avoid recomputing the same prefixes for different requests, the service maintains a persistent KV cache.

Each request is represented by a string of lowercase Latin letters, where every letter denotes one token. KV cache entries are shared by requests with common prefixes. We model the current contents of the cache as a trie.

Initially, the trie contains only the root and has no edges.

When processing a request string s, the following operations are performed in order:

1. Soy processes the entire string from left to right, starting at the root of the trie.
2. If the edge corresponding to the next token already exists, its cached result is reused at no cost.
3. Otherwise, Soy must compute the corresponding KV state. This costs 1 unit of computation, and the missing edge is added to the trie.

The entire request is processed before any cache entry is removed. In particular, all missing edges of the request are first added, even if this temporarily makes the trie contain more than m edges.

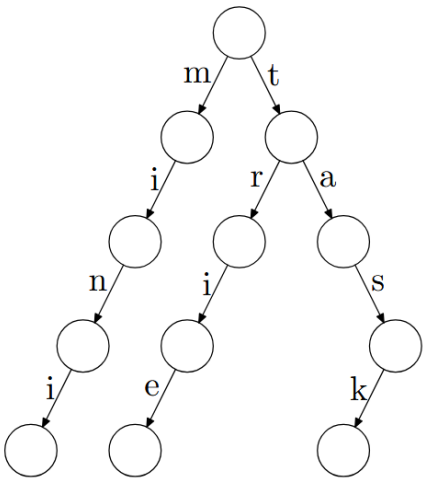
After the whole request has been processed, Soy applies the persistent-cache limit. If the trie contains more than m edges, he must repeatedly delete a leaf vertex together with the edge connecting it to its parent until exactly m edges remain. A leaf is a vertex with no children in the current trie. Edges added while processing the current request may also be deleted during this step.

Soy knows the complete sequence of n requests in advance. For every pruning step, he may choose which leaf edges to delete.

Determine the minimum possible total computation cost required to process all n requests.

Trie is a data structure that stores a set of strings as a rooted tree. The tree has the following structure. Each edge of the tree is labeled with a single letter, there are no two edges labeled with the same letter that come out of each node. Each string can be read by walking along some path from the root to some vertex.

For instance, we can build a trie for strings "`min`", "`trie`", "`task`", and "`mini`", and it looks like this:



Input

The first line of each test contains two integers n and m ($1 \le n \le 10^6$, $1 \le m \le 10^9$).

The i-th of the following n lines of each test contains one string s_i ”— the sequence of tokens in the i-th request. It is guaranteed that s_i only consists of lowercase Latin letters. It is guaranteed that the sum of $|s_i|$ does not exceed 10^6.

Output

Output one integer ”— the minimum total computation cost across all n requests.

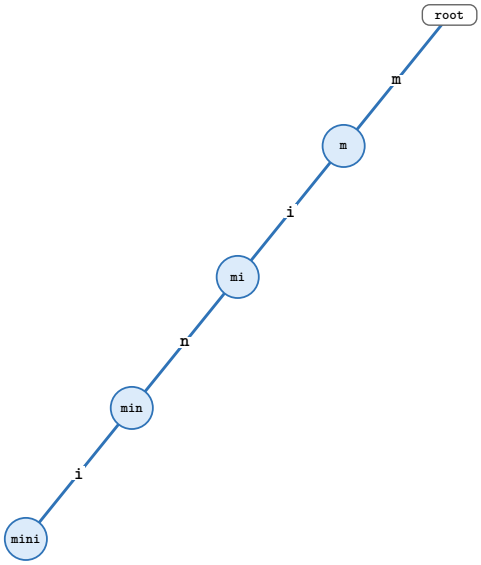
Example

standard input	standard output
4 4 mini trie task min	11

Note

Blue: newly computed and kept **Green**: reused on this request
Gray: cached after pruning **Red dashed**: removed after this request

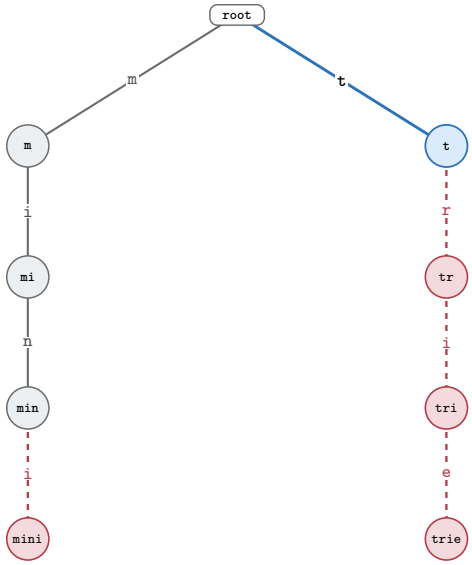
Request 1: `mini` - cost +4



All four edges are computed, and no pruning is needed.

Cache after pruning (4/4): `m`, `mi`, `min`, `mini`.
Cumulative cost: 4.

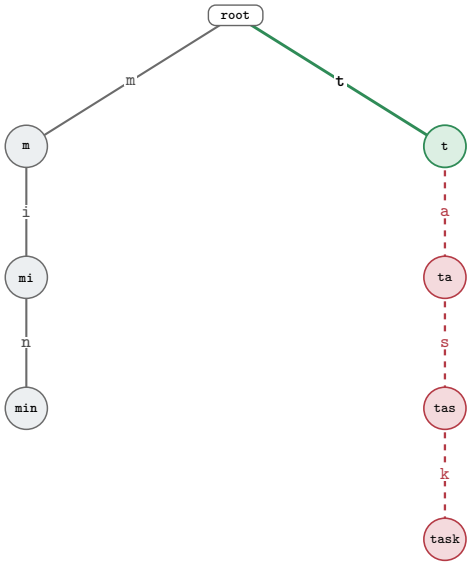
Request 2: `trie` - cost +4



All four request edges are computed; the dashed edges are then removed.

Cache after pruning (4/4): `m`, `mi`, `min`, `t`.
Cumulative cost: $4 + 4 = 8$.

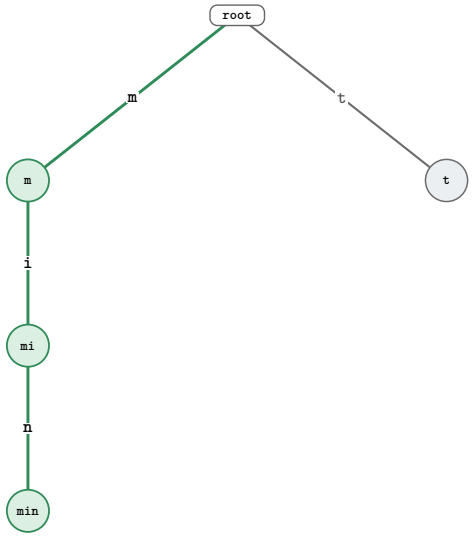
Request 3: `task` - cost +3



The edge `t` is reused; the three computed suffix edges are removed.

Cache after pruning (4/4): `m`, `mi`, `min`, `t`.
Cumulative cost: $4 + 4 + 3 = 11$.

Request 4: `min` - cost +0



The complete request path is reused; the cache does not change.

Cache after pruning (4/4): `m`, `mi`, `min`, `t`.
Total cost: $4 + 4 + 3 + 0 = \boxed{11}$.

This page is intentionally left blank.